

USEREF

SIMPLE COUNTER

```
import { useState } from 'react'
import './App.css'

function App() {
  const [count, setCount] = useState(0);

  function handleIncrement() {
    setCount(count + 1);
  }

  return (
    <div>
      <button onClick={handleIncrement}>
        Increment
      </button>
      <br/>

      <div>
        Count: {count}
      </div>
    </div>
  )
}

export default App
```

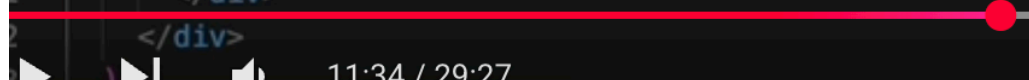
CASE WHEN VALUE OF VARIABLE IS NOT INCREMENTING

```
1
2 import { useState,useEffect } from 'react'
3 import './App.css'
4
5 function App() {
6   const [count, setCount] = useState(0);
7   let val = 1;
8
9   function handleIncrement(){
10     val = val + 1;
11     console.log("Value of val: ", val);
12     setCount(count +1);
13   }
14
15   //it runs on every render
16   useEffect(() => {
17     console.log("main ferse render hogya hu")
18   })
19
20
21   return (
22     <div>
23
24       <button onClick={handleIncrement}>
25         Increment
26       </button>
27       <br/>
28
29       <div>
30         Count: {count}
31       </div>
32     </div>
  
```

HERE , ON EVERY CLICK , UI IS RERENDERED , THAT IS WHY , VAL BECOMES 1 AGAIN AND JUST INCREMENTS TO TWO

CASE WHEN VALUE INCREMENTS CORRECTLY

```
1
2 import { useState,useEffect, useRef } from 'react'
3 import './App.css'
4
5 function App() {
6   ⚠ const [count, setCount] = useState(0);
7   let val = useRef(0);
8
9   function handleIncrement() {
10     val.current = val.current + 1;
11     console.log("Value of val: ", val.current);
12     setCount(count +1);
13   }
14
15   //it runs on every render
16   useEffect(() => {
17     console.log("main ferse render hogya hu")
18   })
19
20   return (
21     <div>
22       <button onClick={handleIncrement}>
23         Increment
24       </button>
25       <br/>
26       <div>
27         Count: {count}
28       </div>
29     </div>
30   )
31 }
```



.CURRENT IS USED TO EXCESS THE REFERENCE CREATED

ATTATCHING REFERECE TO A ELEMENT

```
let btnRef = useRef();

function changeColor() {
  btnRef.current.style.backgroundColor = "red";
}

return (
  <div>
    <button
      ref={btnRef}
      onClick={handleIncrement}>
      Increment
    </button>
    <br/>
  </div>
  <button onClick={changeColor}>
    Change Color of 1st Button
  </button>
  <br/>
</div>
```

PEHLE EK REF BANAYA , FIR USS REF KO ATTATCH KIYA ,
EK BUTTON KE SAATH , AB .CURRENT MEIN HUM USS REF
WALI CHEEZ KO MANIPULATE KAR SAKTE HEIN .

COMPLEX PROBLEM

```
const [time, setTime] = useState(0);

let timerRef = useRef(null);

function startTimer() {
  timerRef.current = setInterval(() => {
    setTime(time => time+1)
  }, 1000);
}

function stopTimer() {
  clearInterval(timerRef.current);
  timerRef.current = null;
}

return (
  <div>
    <h1>StopWatch: {time} seconds</h1>
    <button onClick={startTimer}>
      Start
    </button>
    <br/> <br/>
  </div>
)
```

YAHA PER HUM TIMEREF.CURR KE ANDER VALUE STORE KARWA RAHE HEIN TAAKI USKO CLER karne ke time pe excess kar sake ,
REMEMBER — YEH JO SETINTERVAL HAI TIME KI VALUE KO HI RETURN KARTA HAI
FIR HUM USKO STOP TIMER MEIN DESTROY KAR RAHE HEIN..

####

WHENEVER STATE VARIABLE CHANGES , WHOLE CODE IS RE RENDERED . USEREF HELPS US OVERCOME THIS PROBLEM .

####

WHEN WE GIVE REF TO A CERTAIN VARIABLE , EVEN IF THE WHOLE UI IS RENDERED AGAIN , THE PREVIOUS VALUE OF VARIABLE IS MANIPULATED

##

HERE YOU DO NOT NEED ANY KIND OF STATE FUNCTION

##

USE STATE STATE MAINTAIN KAR RAHA THA , USING STATE FUNCTION

##

HERE WE DO NOT NEED STATE FUNCTION

##

LETS SAY AGAR MEIN EK REF CREATE KAR DU AUR LIKH DU KI IN CLICK VALUE.CURRENT. = VALUE.CURRENT + 1 ; AUR VALUE.CURRENT KO {} KARLU ... TOH BINA RERENDER KE APNA INCREMENTER CREATE HUA HAI

##

USESTATE MEIN VALUE KI PERSISTANCE KE LIYE STATE FUNCTION CREATE KARNA PADTA HAI AUR HAR STATE VARIABLE CHANGE PE RERENDERING OF UI HOTI HAI

##

USEREF MEIN BAS EK VARIABLE KO REF BOLDOA UR USKI VALUE KO .CURRENT USE KARKE XHANGE KAR SAKTE HEIN .

